

Analysis Of Algorithm

NAME :- AHMED ALI _____ INSTRUCTOR :- SIR USMAN SHARIF

S A P :- 6 5 2 3 5 _____ Assignment 03

Comparing Different Sorting Techniques

QUESTION 2 :- For each algorithm, test with 4 different arrays:

Array Size Order Example :-

5 Sorted [1, 2, 3, 4, 5] , 5 Reverse Sorted [5, 4, 3, 2, 1] , 100

Sorted [1, 2, ..., 100], 100 Reverse Sorted [100, 99, ..., 1]

Total runs = 4 algorithms × 4 cases = 16 runs.

4 Codes Of Bubble Sort

Code 1:- 5 Sorted Element

```
#include <iostream>
#include <ctime>
using namespace std;
int main()
{
    int arr[5] = {1, 2, 3, 4, 5};
    clock_t start = clock();
    for(int i = 0; i < 4; i++)
    {
        for(int j = 0; j < 4 - i; j++)
```

```
{
    if(arr[j] > arr[j + 1])
    {
        swap(arr[j], arr[j + 1]);    }    }    }
clock_t end = clock();
cout << "Sorted Array:\n";
for(int i = 0; i < 5; i++)
    cout << arr[i] << " ";
double time_taken =
    ((double)(end - start) / CLOCKS_PER_SEC) * 1000000000;
cout << "\n\nTime Taken: " << time_taken << " nanoseconds";
return 0;
}
```

Code 2:- 5 UNSorted Element

```
#include <iostream>
#include <ctime>
using namespace std;
int main()
{
    int arr[5] = {5, 4, 3, 2, 1};
    clock_t start = clock();
    for(int i = 0; i < 4; i++)
    {
        for(int j = 0; j < 4 - i; j++)
        {
            if(arr[j] > arr[j + 1])
            {
                swap(arr[j], arr[j + 1]);    }
        }
    }

    clock_t end = clock();
    cout << "Sorted Array:\n";
    for(int i = 0; i < 5; i++)
```

```
    cout << arr[i] << " ";  
  
    double time_taken =  
    ((double)(end - start) / CLOCKS_PER_SEC) * 1000000000;  
  
    cout << "\n\nTime Taken: " << time_taken << " nanoseconds";  
  
    return 0; }
```

Code 3 :- 100 Sorted Element

```
#include <iostream>  
  
#include <ctime>  
  
using namespace std;  
  
int main()  
{   int arr[100] = {  
1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,34,35,36,37,38,39,  
40,41,42,43,44,45,46,47,48,49,50,51,52,53,54,55,56,57,58,59,60,61,62,63,64,65,66,67,68,69,70,71,72,73,74,7  
5,76,77,78,79,80,81,82,83,84,85,86,87,88,89,90,91,92,93,94,95,96,97,98,99,100 };  
  
    clock_t start = clock();  
  
    for(int i = 0; i < 99; i++) {  
        for(int j = 0; j < 99 - i; j++) {  
            if(arr[j] > arr[j + 1]) {  
                swap(arr[j], arr[j + 1]); } } }  
  
    clock_t end = clock();  
  
    cout << "Sorted Array:\n";  
  
    for(int i = 0; i < 100; i++)  
        cout << arr[i] << " ";  
  
    double time_taken =  
    ((double)(end - start) / CLOCKS_PER_SEC) * 1000000000;  
  
    cout << "\n\nTime Taken: " << time_taken << " nanoseconds";  
  
    return 0; }
```

Code 4 :- 100 UNSorted Element :-

```
#include <iostream>  
  
#include <ctime>  
  
using namespace std;  
  
int main() {  
    int arr[100] = {
```

```
100,99,98,97,96,95,94,93,92,91,90,89,88,87,86,85,84,83,82,81,80,79,78,77,76,75,74,73,72,71,70,69,68,67,66,
65,64,63,62,61,60,59,58,57,56,55,54,53,52,51,50,49,48,47,46,45,44,43,42,41,40,39,38,37,36,35,34,33,32,31,3
0,29,28,27,26,25,24,23,22,21, 20,19,18,17,16,15,14,13,12,11, 10,9,8,7,6,5,4,3,2,1  };
```

```
clock_t start = clock();
for(int i = 0; i < 99; i++) {
    for(int j = 0; j < 99 - i; j++) {
        if(arr[j] > arr[j + 1]) {
            swap(arr[j], arr[j + 1]);
        } } }
clock_t end = clock();
cout << "Sorted Array:\n";
for(int i = 0; i < 100; i++)
    cout << arr[i] << " ";
double time_taken =
((double)(end - start) / CLOCKS_PER_SEC) * 1000000000;
cout << "\n\nTime Taken: " << time_taken << " nanoseconds
return 0;
}
```

4 Codes Of Selection Sort

Code 1 :- 5 Sorted Element :-

```
#include <iostream>
#include <ctime>
using namespace std;
int main()
{ int arr[5] = {1, 2, 3, 4, 5};
    clock_t start = clock();
    for(int i = 0; i < 4; i++) {
        int min = i;

        for(int j = i + 1; j < 5; j++) {
            if(arr[j] < arr[min])
                min = j;
        }
    }
}
```

```
        swap(arr[i], arr[min]); }  
clock_t end = clock();  
cout << "Sorted Array:\n";  
for(int i = 0; i < 5; i++)  
    cout << arr[i] << " ";  
double time_taken = ((double)(end - start) / CLOCKS_PER_SEC) * 1000000000;  
cout << "\n\nTime Taken: " << time_taken << " nanoseconds";  
return 0; }
```

Code 2 :- 5 UNSorted Element :-

```
#include <iostream>  
#include <ctime>  
using namespace std;  
int main()  
{   int arr[5] = {5, 4, 3, 2, 1};  
    clock_t start = clock();  
    for(int i = 0; i < 4; i++) {  
        int min = i;  
        for(int j = i + 1; j < 5; j++) {  
            if(arr[j] < arr[min])  
                min = j;    }  
        swap(arr[i], arr[min]); }  
    clock_t end = clock();  
    cout << "Sorted Array:\n";  
    for(int i = 0; i < 5; i++)  
        cout << arr[i] << " ";  
    double time_taken =  
        ((double)(end - start) / CLOCKS_PER_SEC) * 1000000000;  
    cout << "\n\nTime Taken: " << time_taken << " nanoseconds";  
    return 0; }
```

Code 3 :- 100 Sorted Element :-

```
#include <iostream>  
#include <ctime>  
using namespace std;  
int main()
```

```
{
    int arr[100] = {
        1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,34,35,36,37,38,39,
        40,41,42,43,44,45,46,47,48,49,50,51,52,53,54,55,56,57,58,59,60,61,62,63,64,65,66,67,68,69,70,71,72,73,74,7
        5,76,77,78,79,80,81,82,83,84,85,86,87,88,89,90,91,92,93,94,95,96,97,98,99,100    };

    clock_t start = clock();

    for(int i = 0; i < 99; i++)
    {
        int min = i;

        for(int j = i + 1; j < 100; j++)    {

            if(arr[j] < arr[min])

                min = j;        }

        swap(arr[i], arr[min]);    }

    clock_t end = clock();

    cout << "Sorted Array:\n";

    for(int i = 0; i < 100; i++)

        cout << arr[i] << " ";

    double time_taken =

        ((double)(end - start) / CLOCKS_PER_SEC) * 1000000000;

    cout << "\n\nTime Taken: " << time_taken << " nanoseconds";

    return 0; }
```

Code 4 :- 100 UNSorted Element :-

```
#include <iostream>

#include <ctime>

using namespace std;

int main()

{

    int arr[100] = {

        100,99,98,97,96,95,94,93,92,91,90,89,88,87,86,85,84,83,82,81,80,79,78,77,76,75,74,73,72,71,70,69,68,67,66,
        65,64,63,62,61,60,59,58,57,56,55,54,53,52,51,50,49,48,47,46,45,44,43,42,41,40,39,38,37,36,35,34,33,32,31,3
        0,29,28,27,26,25,24,23,22,21, 20,19,18,17,16,15,14,13,12,11, 10,9,8,7,6,5,4,3,2,1    };

    clock_t start = clock();

    for(int i = 0; i < 99; i++) {

        int min = i;

        for(int j = i + 1; j < 100; j++)    {

            if(arr[j] < arr[min])

                min = j;    }
```

```
        swap(arr[i], arr[min]); }  
clock_t end = clock();  
cout << "Sorted Array:\n";  
for(int i = 0; i < 100; i++)  
    cout << arr[i] << " ";  
double time_taken =  
((double)(end - start) / CLOCKS_PER_SEC) * 1000000000;  
cout << "\n\nTime Taken: " << time_taken << " nanoseconds";  
return 0;}
```

4 Codes Of Quick Sort

Code 1 :- 5 Sorted Element :-

```
#include <iostream>  
#include <ctime>  
using namespace std;  
int main()  
{ int arr[5] = {1, 2, 3, 4, 5};  
  clock_t start = clock();  
  int low = 0, high = 4;  
  int stack[10];  
  int top = -1;  
  stack[++top] = low;  
  stack[++top] = high;  
  while(top >= 0)  
  {   high = stack[top--];  
      low = stack[top--];  
  
      int i = low, j = high;  
      int pivot = arr[(low + high) / 2];  
      while(i <= j)  
      {   while(arr[i] < pivot) i++;  
          while(arr[j] > pivot) j--;
```

```
        if(i <= j)    {
            swap(arr[i], arr[j]);
            i++; j--;    }    }
    if(low < j)    {
        stack[++top] = low;
        stack[++top] = j;    }
    if(i < high)    {
        stack[++top] = i;
        stack[++top] = high;    }    }
    clock_t end = clock();
    cout << "Sorted Array:\n";
    for(int i = 0; i < 5; i++)
        cout << arr[i] << " ";
    double time_taken =
    ((double)(end - start) / CLOCKS_PER_SEC) * 1000000000;
    cout << "\n\nTime Taken: " << time_taken << " nanoseconds";
    return 0; }
```

Code 2 :- 5 UNSorted Element :-

```
#include <iostream>
#include <ctime>
using namespace std;
int main()
{    int arr[5] = {5, 4, 3, 2, 1};
    clock_t start = clock();
    int low = 0, high = 4;
    int stack[10];
    int top = -1;
    stack[++top] = low;
    stack[++top] = high;

    while(top >= 0)
    {    high = stack[top--];
        low = stack[top--];
        int i = low, j = high;
```



```
int pivot = arr[(low + high) / 2];

while(i <= j) {
    while(arr[i] < pivot) i++;
    while(arr[j] > pivot) j--;
    if(i <= j) {
        swap(arr[i], arr[j]);
        i++; j--;
    }
    if(low < j) {
        stack[++top] = low;
        stack[++top] = j;
    }
    if(i < high) {
        stack[++top] = i;
        stack[++top] = high;
    }
}

clock_t end = clock();
cout << "Sorted Array:\n";
for(int i = 0; i < 5; i++)
    cout << arr[i] << " ";
double time_taken =
((double)(end - start) / CLOCKS_PER_SEC) * 1000000000;
cout << "\n\nTime Taken: " << time_taken << " nanoseconds";
return 0; }
```

Code 3 :- 100 Sorted Element :-

```
#include <iostream>
#include <ctime>
using namespace std;
int main()
{
    int arr[100] = {
        1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,34,35,36,37,38,39,
        40,41,42,43,44,45,46,47,48,49,50,51,52,53,54,55,56,57,58,59,60,61,62,63,64,65,66,67,68,69,70,71,72,73,74,7
        5,76,77,78,79,80,81,82,83,84,85,86,87,88,89,90,91,92,93,94,95,96,97,98,99,100};
    clock_t start = clock();
    int low = 0, high = 99;
    int stack[200];
    int top = -1;
    stack[++top] = low;
```

```
stack[++top] = high;
while(top >= 0) {
    high = stack[top--];
    low = stack[top--];
    int i = low, j = high;
    int pivot = arr[(low + high) / 2];
    while(i <= j) {
        while(arr[i] < pivot) i++;
        while(arr[j] > pivot) j--;
        if(i <= j) {
            swap(arr[i], arr[j]);
            i++; j--;
        }
        if(low < j) {
            stack[++top] = low;
            stack[++top] = j;
        }
        if(i < high) {
            stack[++top] = i;
            stack[++top] = high;
        }
    }
    clock_t end = clock();
    cout << "Sorted Array:\n";
    for(int i = 0; i < 100; i++)
        cout << arr[i] << " ";
    double time_taken =
        ((double)(end - start) / CLOCKS_PER_SEC) * 1000000000;
    cout << "\n\nTime Taken: " << time_taken << " nanoseconds";
    return 0;}
```

Code 4 :- 100 UN Sorted Element :-

```
#include <iostream>
#include <ctime>
using namespace std;
int main()
{
    int arr[100] =
    {100,99,98,97,96,95,94,93,92,91,90,89,88,87,86,85,84,83,82,81,80,79,78,77,76,75,74,73,72,71,70,69,68,67,66,
```

65,64,63,62,61,60,59,58,57,56,55,54,53,52,51,50,49,48,47,46,45,44,43,42,41,40,39,38,37,36,35,34,33,32,31,30,29,28,27,26,25,24,23,22,21, 20,19,18,17,16,15,14,13,12,11, 10,9,8,7,6,5,4,3,2,1};

```
clock_t start = clock();

int low = 0, high = 99;

int stack[200];

int top = -1;

stack[++top] = low;
stack[++top] = high;

while(top >= 0)
{
    high = stack[top--];
    low = stack[top--];

    int i = low, j = high;

    int pivot = arr[(low + high) / 2];

    while(i <= j) {
        while(arr[i] < pivot) i++;
        while(arr[j] > pivot) j--;

        if(i <= j) {
            swap(arr[i], arr[j]);
            i++; j--;
        }
    }

    if(low < j) {
        stack[++top] = low;
        stack[++top] = j;
    }

    if(i < high) {
        stack[++top] = i;
        stack[++top] = high;
    }
}

clock_t end = clock();

cout << "Sorted Array:\n";

for(int i = 0; i < 100; i++)
    cout << arr[i] << " ";

double time_taken =
((double)(end - start) / CLOCKS_PER_SEC) * 1000000000;

cout << "\n\nTime Taken: " << time_taken << " nanoseconds";

return 0; }
```

4 Codes Of Merge Sort

Code 1 :- 5 Sorted Element :-

```
#include <iostream>

#include <ctime>

using namespace std;

int main()

{   int arr[5] = {1, 2, 3, 4, 5};

    int temp[5];

    clock_t start = clock();

    for(int size = 1; size < 5; size *= 2)   {

        for(int left = 0; left < 5 - size; left += 2 * size)    {

            int mid = left + size - 1;

            int right = min(left + 2 * size - 1, 4);

            int i = left, j = mid + 1, k = left;

            while(i <= mid && j <= right)

                temp[k++] = (arr[i] <= arr[j]) ? arr[i++] : arr[j++];

            while(i <= mid) temp[k++] = arr[i++];

            while(j <= right) temp[k++] = arr[j++];

            for(int p = left; p <= right; p++)

                arr[p] = temp[p];        }   }

    clock_t end = clock();

    cout << "Sorted Array:\n";

    for(int i = 0; i < 5; i++)

        cout << arr[i] << " ";

    double time_taken =

        ((double)(end - start) / CLOCKS_PER_SEC) * 1000000000;

    cout << "\n\nTime Taken: " << time_taken << " nanoseconds";

    return 0;}
```

Code 2 :- 5 UNSorted Element :-

```
#include <iostream>
```

```
#include <ctime>

using namespace std;

int main() {

    int arr[5] = {5, 4, 3, 2, 1};

    int temp[5];

    clock_t start = clock();

    for(int size = 1; size < 5; size *= 2) {

        for(int left = 0; left < 5 - size; left += 2 * size) {

            int mid = left + size - 1;

            int right = min(left + 2 * size - 1, 4);

            int i = left, j = mid + 1, k = left;

            while(i <= mid && j <= right)

                temp[k++] = (arr[i] <= arr[j]) ? arr[i++] : arr[j++];

            while(i <= mid) temp[k++] = arr[i++];

            while(j <= right) temp[k++] = arr[j++];

            for(int p = left; p <= right; p++)

                arr[p] = temp[p]; } }

    clock_t end = clock();

    cout << "Sorted Array:\n";

    for(int i = 0; i < 5; i++)

        cout << arr[i] << " ";

    double time_taken =

        ((double)(end - start) / CLOCKS_PER_SEC) * 1000000000;

    cout << "\n\nTime Taken: " << time_taken << " nanoseconds";

    return 0;}
```

Code 3 :- 100 Sorted Element :-

```
#include <iostream>

#include <ctime>

using namespace std;

int main(){

    int arr[100] = {

        1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,34,35,36,37,38,39,

        40,41,42,43,44,45,46,47,48,49,50,51,52,53,54,55,56,57,58,59,60,61,62,63,64,65,66,67,68,69,70,71,72,73,74,7

        5,76,77,78,79,80,81,82,83,84,85,86,87,88,89,90,91,92,93,94,95,96,97,98,99,100 };

    int temp[100];
```

```

clock_t start = clock();
for(int size = 1; size < 100; size *= 2) {
    for(int left = 0; left < 100 - size; left += 2 * size) {
        int mid = left + size - 1;
        int right = min(left + 2 * size - 1, 99);
        int i = left, j = mid + 1, k = left;
        while(i <= mid && j <= right)
            temp[k++] = (arr[i] <= arr[j]) ? arr[i++] : arr[j++];
        while(i <= mid) temp[k++] = arr[i++];
        while(j <= right) temp[k++] = arr[j++];
        for(int p = left; p <= right; p++)
            arr[p] = temp[p];    } }
clock_t end = clock();
cout << "Sorted Array:\n";
for(int i = 0; i < 100; i++)
    cout << arr[i] << " ";
double time_taken =
((double)(end - start) / CLOCKS_PER_SEC) * 1000000000;
cout << "\n\nTime Taken: " << time_taken << " nanoseconds";
return 0;}

```

Code 4:- 100 UN Sorted Element :-

```

#include <iostream>

#include <ctime>

using namespace std;

int main()
{
    int arr[100] = {
        100,99,98,97,96,95,94,93,92,91,90,89,88,87,86,85,84,83,82,81,80,79,78,77,76,75,74,73,72,71,70,69,68,67,66,
        65,64,63,62,61,60,59,58,57,56,55,54,53,52,51,50,49,48,47,46,45,44,43,42,41,40,39,38,37,36,35,34,33,32,31,3
        0,29,28,27,26,25,24,23,22,21, 20,19,18,17,16,15,14,13,12,11, 10,9,8,7,6,5,4,3,2,1    };

    int temp[100];

    clock_t start = clock();

    for(int size = 1; size < 100; size *= 2) {
        for(int left = 0; left < 100 - size; left += 2 * size) {
            int mid = left + size - 1;

```

```

int right = min(left + 2 * size - 1, 99);

int i = left, j = mid + 1, k = left;

while(i <= mid && j <= right)

    temp[k++] = (arr[i] <= arr[j]) ? arr[i++] : arr[j++];

while(i <= mid) temp[k++] = arr[i++];

while(j <= right) temp[k++] = arr[j++];

for(int p = left; p <= right; p++)

    arr[p] = temp[p];    } }

clock_t end = clock();

cout << "Sorted Array:\n";

for(int i = 0; i < 100; i++)

    cout << arr[i] << " ";

double time_taken =

((double)(end - start) / CLOCKS_PER_SEC) * 1000000000;

cout << "\n\nTime Taken: " << time_taken << " nanoseconds";

return 0; }

```

TIME TAKEN FOR COMPARISON OG 4 SORTING TECHNIQUES

Bubble Sort

Input Case	Run 1	Run 2	Run 3	Average
5 Sorted	1000	2000	1000	1333
5 Reverse	2000	2000	3000	2333
100 Sorted	14000	15000	16000	15000
100 Reverse	67000	54000	8000	43000

Selection Sort

Input Case	Run 1	Run 2	Run 3	Average
5 Sorted	1000	1000	2000	1333
5 Reverse	2000	1000	1000	1333
100 Sorted	12000	13000	14000	13000
100 Reverse	22000	32000	19000	24333

Quick Sort

Input Case	Run 1	Run 2	Run 3	Average
5 Sorted	1000	1000	2000	1333
5 Reverse	1000	1000	3000	1667
100 Sorted	4000	4000	5000	4333
100 Reverse	6000	8000	9000	7667

Merge Sort

Input Case	Run 1	Run 2	Run 3	Average
5 Sorted	1000	1000	1000	1000
5 Reverse	1000	2000	1000	1333
100 Sorted	5000	5000	6000	5333
100 Reverse	7000	8000	11000	8667

SPACE COMPLEXITY

Space Complexity

Algorithm	Auxiliary Space	In-Place or Extra Arrays	Explanation
Bubble Sort	$O(1)$	In-Place	Sorts elements within the same array and does not require extra memory
Selection Sort	$O(1)$	In-Place	Uses only a temporary variable for swapping elements
Quick Sort	$O(\log n)$	In-Place	Uses recursive call stack during partitioning
Merge Sort	$O(n)$	Needs Extra Arrays	Requires temporary arrays during the merging process

SWAPS COUNT / COMPARISON

Algorithm	Comparisons	Swaps	Explanation
Bubble Sort	High	High	Performs many comparisons and swaps, especially for reverse-sorted arrays
Selection Sort	High	Low	Performs many comparisons but fewer swaps compared to Bubble Sort
Quick Sort	Moderate	Moderate	Efficient partitioning reduces unnecessary comparisons and swaps
Merge Sort	Moderate	No Direct Swaps	Uses merging instead of swapping elements directly

ANALYSIS

Analysis

The current experiment involves four sorting algorithms – Bubble Sort, Selection Sort, Quick Sort, and Merge Sort. These four sorting algorithms are used to analyze and sort data, where input data can differ in its input number (5, 100), as well as in order of elements (sorted array, reverse sorted array). The experiment helps observe sorting algorithms' behavior and compare theoretical complexity with actual result.

Which algorithm works faster for input size 5? For 100?

In the case of 5 input numbers, each sorting algorithm has been able to finish task almost instantly. In particular, it has been hard to tell one algorithm from the other because of extremely short difference in completion time. On the other hand, Quick Sort and Merge Sort have been noticeably faster than other two algorithms. The reason is that these two sorting algorithms use divide-and-conquer approach leading to reduced number of comparisons.

The case with 100 inputs has been different since Quick Sort has been considerably faster than all other sorting algorithms, which proves $O(n \log n)$ complexity. At the same time, all three Merge Sort test cases have been faster than Bubble Sort and Selection Sort due to smaller number of comparisons.

Effect of sorted/reverse-sorted input :-

Input had a noticeable influence on the performance of some algorithms.

Bubble Sort was very efficient with pre-sorted data, yet, performed poorly with reversed sort due to having the maximum amount of swaps for such a situation.

There were no considerable differences for Selection Sort with any kind of input since this algorithm works through the whole array trying to find the smallest element.

Performance of Quick Sort relied upon the pivot point, yet was efficient in both scenarios. Merge Sort had exactly the same execution time regardless of input types due to algorithm consistency.

Are obtained results in accordance with theoretical complexity?

- Bubble Sort and Selection Sort have $O(n^2)$ complexity, therefore, are inefficient when executing with 100 elements unlike 5.
- Both Quick Sort and Merge Sort exhibit $O(n \log n)$ complexity, thus perform excellently with more complex input.
- Reverse-sorted input had a bad impact on Bubble Sort execution as predicted by theory.
- Selection Sort was indifferent to sorted input, as was predicted theoretically.
- Quick Sort scaled well with larger number of elements as predicted, demonstrating average case complexity.
- Merge Sort demonstrated consistent execution time, just as predicted by theory.

Which Algorithm Uses More Memory?

- The space complexity of the bubble sort algorithm is constant or $O(1)$.
- This implies that the process of sorting would be done using the same array.
- The space complexity of the selection sort algorithm is constant or $O(1)$.
- This implies that no additional memory would be used by the algorithm.
- The space complexity of the quick sort algorithm is logarithmic since the quick sort algorithm is a recursive algorithm.
- The space complexity of the merge sort algorithm is linear since new arrays are generated during the merging process.
- Therefore, merge sort uses more memory compared to bubble sort and selection sort algorithms.

Conclusion

All sorting algorithms perform effectively and efficiently in sorting smaller volumes of data; hence it does not matter which one to use. However, when sorting larger volumes of data, quick sort method is highly efficient.

As far as memory is concerned, the two algorithms that consume the least amount of memory are Bubble Sort and Selection Sort because of their constant memory requirements. Quick Sort requires moderate memory owing to recursion, whereas Merge Sort requires the largest amount of memory.